Zcash Name Service

A Deterministic Name Registry over the Orchard Transaction Log

craftsoldier
github.com/craftsoldier julian@zcash.me

Protocol draft — August 25, 2026

Status: Draft
Category: Standards Track
Created: August 25, 2026
License: MIT
Discussion: github.com/craftsoldier/zns-whitepaper

Abstract

Zcash payments use encoded addresses, but Zcash consensus does not assign human-readable names to them. Existing on-chain naming systems cannot be ported to Zcash: transparent registries such as Namecoin publish the namespace in the clear, leaking the address graph Zcash exists to hide; contract-based registries such as ENS require general-purpose on-chain state that Zcash consensus does not provide. The Zcash Name Service (ZNS) maps each Zcash name to a Unified Address over the ordinary Zcash transaction log: each binding is recorded in the memo of an Orchard action, decryptable with the Name Note account's published full viewing key, and anchored by an Orchard note commitment derived from the same tuple. Zcash consensus supplies canonical order and finality; a deterministic reducer derives the registry; the bound Unified Address inherits Orchard shielding. Authorization is delivered by an attested Mint; a future off-chain zero-knowledge name circuit would replace that assumption with a proof.

Status. This document is a work in progress. Sections explicitly marked Open or Provisional are not interoperability requirements. Sections

marked FIXED state normative requirements backed by the cited assumptions. An implementation cannot infer a protocol rule from an unresolved item. The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are to be interpreted as described by BCP 14 only when they appear in uppercase [5].

1 Motivation

Zcash addresses are long encoded strings. A naming system maps these strings to human-readable labels, but this requires enforcing rules about which names are claimed and who controls them. Zcash consensus tracks unspent coins. Lacking a virtual machine to run logic and a global state tree to store variables, it cannot evaluate a naming policy. To bypass this limitation, ZNS separates the execution of the naming policy from the permanent public record. A single party, the Mint, executes the policy off-chain. When a request passes, the Mint writes the resulting name binding into the 512-byte memo field of a shielded Zcash transaction. Zcash consensus seals the transaction into a block, locking the binding into a permanent, chronological order. A Resolver looking up a name downloads the Zcash block headers, uses a published viewing key to decrypt the Mint's memos, and reads the bindings in the exact order consensus finalized them. The Mint issues names, but the accumulated work of the Zcash blockchain prevents it from altering or erasing previously recorded bindings.

2 Architecture

To guarantee the globally unique mapping of human-readable names to Zcash addresses, a naming system requires an agreed-upon history of requests to register, modify, or release names, a strict set of rules, and a resulting registry of name bindings. ZNS treats this registry as derived state by using Zcash exclusively for the history of these requests, executing rules off-chain, and binding their accepted results on-chain.

2.1 System Components

The protocol relies on three distinct components:

The Mint. The authoritative off-chain program that evaluates the naming policy. It runs inside a Trusted Execution Environment (TEE), with its authority bound to approved protocol code through remote attestation. It uses its own Zcash node to read chain state independently.

The Name Note. An Ironwood output used to record a name transition on-chain.

The Resolver. A permissionless client that reads the canonical sequence of Zcash transactions, verifies Name Notes and deterministically derives registry state.

2.2 Independence of Facts

The security of the derived state relies on six distinct properties. The Resolver verifies publicly reconstructible properties directly and relies on Mint attestation for policy checks that are not publicly reconstructible.

Zcash validity: The transaction satisfies Zcash consensus.

Mint authorization: The recognized Mint spending key authorized the transaction.

TEE attestation: The recognized Mint identity executed the approved enclave code.

Name Note verification: The transition tuple encoded in a Name Note matches its Ironwood note commitment.

User authorization: The attested Mint enforces the authorization procedure defined in Section 5.

State derivation: Resolvers reading the canonical Zcash chain under identical protocol parameters derive the same registry state.

3 Name Notes

A name transition must remain interpretable after the program that created it is gone. Its public record must therefore bind the transition to the Orchard output that carries it.

Definition. A Name Note is an Orchard output whose memo encodes one

ZNS transition tuple and whose note commitment is derived from that tuple. The tuple binds an action, a name, a Unified Address, and a predecessor commitment.

3.1 Encoded transition

The Name Note memo is a 512-byte field. A parser removes its maximal trailing sequence of zero bytes. The remaining bytes encode one transition in one of the following formats:

- `ZNS:claim:<name>:<ua>:<prev_rcm>`
- `ZNS:update:<name>:<ua>:<prev_rcm>`
- `ZNS:release:<name>::<prev_rcm>`

The colon byte (`0x3a`) separates fields. The `prev_rcm` field is the 64-character lowercase hexadecimal encoding of the predecessor rcm_σ. The first `claim` uses 64 zeroes.

3.2 Committed tuple

The memo is an encoding. The commitment input is the tuple extracted from it. A valid memo maps to $\sigma = (\alpha, n, u, p)$:

α The exact ASCII bytes `claim`, `update`, or `release`.
n The raw bytes of the `name` field.
u The raw bytes of the `ua` field (empty for `release`).
p The 32 raw bytes decoded from `prev_rcm`.

3.3 Commitment inputs

To prevent ambiguous concatenation, define `LP(x)` as the four-byte unsigned little-endian byte length of `x`, followed by `x` itself. Define the protocol domain tag T as the 12 ASCII bytes `ZcashName/v1`. Quotation marks are not part of T.

For field tag t, define the domain-separated hash

$$H_t(\sigma) = \text{BLAKE2b-512}(\,\text{LP}(T) \parallel \text{LP}(t) \parallel \text{LP}(\alpha) \parallel \text{LP}(n) \parallel \text{LP}(u) \parallel p).$$

The final p is exactly 32 raw bytes and has no length prefix. Each H_t invocation uses unkeyed BLAKE2b-512 with a 64-byte output and no additional personalization. The field tag is the ASCII byte string `rcm` or `psi`.

The two derived values are

$$\mathsf{rcm}_\sigma = \mathsf{ToScalar}(H_{\texttt{rcm}}(\sigma)),$$
$$\psi_\sigma = \mathsf{ToBase}(H_{\texttt{psi}}(\sigma)).$$

The value rcm_σ is the Pallas scalar-field element used as the Orchard note-commitment randomness. The value ψ_σ is the Pallas base-field element included in the Orchard note commitment. ToScalar, ToBase, and the field types are those defined for Orchard by the Zcash Protocol Specification [1, 3]. Neither value is a raw 64-byte BLAKE2b digest; each is the field element obtained by reducing its separately tagged digest. The canonical encoding of either field element is its 32-byte little-endian field representation. ZNS derives both field elements from σ, not from the decrypted `rseed` used by ordinary Orchard receiving [2, 1].

Changing the domain tag, a field tag, field order, length width, byte order, hash parameters, or reduction rule changes the protocol version and requires new test vectors.

3.4 Commitment reconstruction

ZNS replaces the ordinary derivation of ψ and rcm. It does not change the Orchard commitment construction.

A verifier supplies ψ_σ, rcm_σ, and the candidate's remaining note components to the standard Orchard construction [1, 3]. It accepts the Name Note only if the result equals the `cmx` in the on-chain Orchard action.

3.5 Name Note verification

The equality proves one fact: the tuple encoded in the memo is bound to that Orchard output. It does not establish Mint origin, user authorization, policy compliance, or the current state of the namespace.

3.6 Required test vector

For `action=claim`, `name=alice`, `ua=u1xxx`, and a 32-byte zero predecessor, the canonical field encodings are:

Value	Canonical little-endian bytes rendered as hexadecimal
ψ_σ	`bde12553f1d349d9ca6836f6711a256cd353fc2376d5d861324766845bcfbd08`
rcm_σ	`ab91154ea92a1796a0d088e4909ab7f72b0e20896a130c5bb53910044565b020`

This vector tests the commitment primitive only. The string `u1xxx` is not a valid ZIP 316 Unified Address, so this tuple MUST fail candidate validation before state reduction.

Set `g_d` to 32 bytes of `0x11` and `pk_d` to 32 bytes of `0x22`. Set the value to zero and `rho` to 32 bytes of `0x33`. The resulting `cmx` is `53accd0df1c569731e8ad4fc8bcb483b953e3713ecc7a95202442daa026c4a02`. An implementation that produces another value is non-conforming.

4 The Mint

A Name Note proves that a transition is bound to an Orchard output. It cannot determine whether the transition was allowed. That decision requires the namespace's current state and its policy.

The Mint is the single authoritative program that makes this decision. It reads ordered requests from Zcash, derives the accepted namespace state, and applies the active policy. For an accepted request, it emits a self-send containing the resulting Name Note.

4.1 Authority bound to policy

The Mint spending key authorizes every accepted transition. Whoever can use that key can create an apparently valid Name Note without applying the policy. The key must therefore be available only to the program that evaluates the policy.

The Mint runs inside a Trusted Execution Environment (TEE). Its remote attestation binds the Mint key to an approved enclave measurement. A Resolver accepts a Mint self-send only when the active manifest approves that measurement.

This replaces trust in the Mint operator with trust in the TEE attestation system and its hardware root of trust.

4.2 Policy input and output

The Mint evaluates each request against the state derived from the accepted Zcash chain. It must use its own Zcash node and its own replay of the namespace history. It must not rely on a third-party state oracle.

For each request, the active policy decides whether to emit a successor Name Note. The policy includes the following constraints:

Namespace policy. The Mint decides whether a name is claimable, live, renewable, or released. It rejects a transition that conflicts with the accepted state.
Economic policy. The Mint calculates the required payment from the requested name and registration interval. It rejects a request without the required payment.
Request validity. The Mint rejects malformed names, invalid Unified Addresses, and transitions prohibited by the active namespace policy.
Authorization. An `update` or `release` requires the OTP exchange defined in Section 5.

4.3 On-chain record

The Resolver must be able to derive the namespace without the Mint. A Name Note therefore carries the complete transition tuple on-chain. The Mint must not replace a tuple field with an off-chain pointer.

The encoded transition must fit in one 512-byte Orchard memo. The Mint rejects a request whose Name Note exceeds that limit. A multi-memo encoding requires an activated ZNSIP.

4.4 The Active Manifest

The Active Manifest fixes the policy parameters and the accepted enclave measurements. These parameters include pricing, registration intervals, namespace restrictions, and OTP configuration.

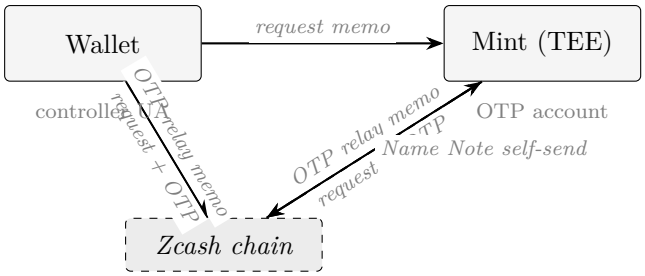
The manifest identifies the immutable source revision and audit report for each measurement. A policy upgrade requires a new approved measurement. The activation and rollback rules are OPEN.

5 User Authorization

Zcash shielded addresses lack a standard message-signing mechanism. Even if a signature scheme existed, a raw signature does not prove temporal liveness or prevent replay attacks without a persistent, on-chain nonce registry.

ZNS solves this with an in-band one-time passcode (OTP) that relies exclusively on Zcash's native encryption. The Mint encrypts a 16-byte random payload to the Unified Address currently on record. Returning that exact payload to the Mint proves two things simultaneously: cryptographic read-access to the address, and temporal liveness bounded by the OTP expiration block.

A `claim` establishes initial state and requires no OTP. An `update` or `release` requires a valid OTP exchange before the Mint creates the Name Note [7].



5.1 Authorization target

Each action uses the following authorization address:

Action	Authorization address	Consequence
`claim`	Proposed Unified Address	The request itself authorizes the claim; no OTP is issued.
`update`	Unified Address in the accepted live tip	The OTP tests access to the address that currently controls the name.
`release`	Unified Address in the accepted live tip	The OTP tests access to the address that currently controls the name.

The active manifest MUST identify the shielded receiver type used for OTP delivery. The Mint selects that receiver from the controller Unified Address under ZIP 316 and rejects an address that does not contain it [4]. A successful OTP response establishes that the requester can receive shielded memos sent to that receiver. It does not establish spending authority, control of every receiver in the Unified Address, or control of an unverified update destination.

The proposed new address in an `update` is not verified by the OTP. A mistyped or attacker-substituted destination can become the live binding if wallet confirmation fails.

The public Name Note account MUST NOT carry OTPs because its full viewing key is public. The Mint sends OTP relay memos from an account whose viewing authority is not published. This separation prevents disclosure through the Name Note viewing key. It does not hide network metadata from the Mint operator, wallet provider, or transaction-submission infrastructure.

5.2 OTP relay memo

The Mint sends the OTP to the selected receiver of the controller Unified Address in an Orchard shielded memo. The relay memo grammar is:

Non-padding bytes
`ZNS:otp:<name>:<verb>:<ua>:<otp>`

The `<verb>` field is `update` or `release`. The `<ua>` field is the target Unified Address from the request. The `<otp>` field is exactly 32 lowercase hexadecimal characters encoding 16 bytes.

The relay memo is exactly 512 bytes and zero-padded. A request parser MUST reject a relay memo as a request memo because its verb is `otp`.

5.3 OTP properties

An OTP is 16 bytes of entropy drawn from a cryptographically secure random source inside the attested environment [7]. The active manifest MUST fix the validity window N in blocks; the current implementation uses $N = 24$. An OTP issued at block h is valid through block $h + N$.

The Mint verifies a provided OTP with constant-time equality, accepts it only for the exact $(\texttt{name}, \texttt{action}, \texttt{ua})$ it was issued for, and burns it on acceptance. A consumed OTP MUST never succeed again. The Mint MUST reject an expired OTP and an OTP for a mismatched transition.

The active manifest MUST fix the validity window, the persistent-state format, and the maximum number of attempts. These values are Open until the manifest fixes them.

5.4 Request binding

An unbound OTP could authorize a substituted request. The Mint binds each OTP to the exact $(\texttt{name}, \texttt{action}, \texttt{ua})$ tuple in the first request. It accepts the OTP only in a second request with the same tuple.

The Mint MAY impose additional request-binding rules such as a minimum request value or a payment locator. Those rules are Mint policy, not independently verifiable by a Resolver.

5.5 Public verification boundary

The OTP exchange does not produce public per-request authorization evidence. A Resolver can verify the resulting Name Note, Mint self-send, and Mint attestation. It cannot reconstruct the OTP exchange from the public Name Note log.

Attestation establishes that a recognized Mint identity runs approved policy under the stated TEE assumptions. It does not disclose a transcript proving that one OTP was delivered and consumed. Per-request public authorization evidence requires a separate proof construction.

6 The Resolver

The Zcash chain records candidate Name Notes. It does not identify which candidates were authorized by the Mint or which transition is current.

The Resolver derives this state from the canonical chain. It is permissionless. Two Resolvers that scan the same canonical chain under the same active manifest must derive the same registry.

6.1 Scanning

Standard Orchard wallets reject Name Notes because ZNS derives ψ and rcm from the transition tuple rather than from the ordinary ZIP-212 `rseed` derivation. A Resolver uses a dedicated scanner and the published Name Note full viewing key.

For each candidate output, the Resolver performs three checks:

Decryption. The Resolver obtains the memo and note components by authenticated decryption, bypassing the standard ZIP-212 receiving check.
Name Note verification. The Resolver parses the memo and reconstructs the Orchard commitment as specified in Section 3.
Mint origin. The Resolver accepts the candidate only if its creating Orchard action spends a note from the recognized Mint-controlled unspent set.

For every accepted Name Note, the Resolver updates the Mint-controlled unspent set and follows the `prev_rcm` link for the affected name. When the best chain changes, it rolls back to the common ancestor and replays the

replacement chain.

7 Failure Domains

ZNS is an overlay network. It achieves consensus without requiring Zcash nodes to evaluate name registry rules. To achieve this, the architecture accepts specific failure domains as calculated trade-offs.

7.1 The Execution Domain: TEE vs. ZKP

A zero-knowledge proof of a state transition is mathematically pure, but enforcing a globally unique namespace requires proving a negative: that a requested name is not currently registered by anyone else.

If a decentralized protocol relies on client-side ZK-SNARKs, the user's device must prove non-inclusion against the global state root. Doing this privately requires Private Information Retrieval (PIR) inside the circuit, which is computationally prohibitive for a mobile wallet. Furthermore, pushing the registry rules into a client-side circuit requires wallet consensus: every major Zcash wallet would have to bundle and execute a heavy, protocol-specific proving circuit.

Conversely, off-chain ZK-Rollups that execute server-side proofs reintroduce the trusted sequencer and rely on infinitely complex, upgradable smart contracts.

ZNS trades cryptographic purity for immediate viability. By isolating the state machine inside a Trusted Execution Environment (TEE), the wallet's job is trivialized to sending a standard Orchard memo. The failure domain shifts entirely to the hardware vendor (e.g., an Intel or AMD side-channel compromise). If the TEE is breached, the attacker can forge Mint signatures and overwrite the registry.

7.2 The Availability Domain: On-Chain vs. Off-Chain

Storing the registry state off-chain and merely anchoring a Merkle root on-chain is standard practice for scalable overlay networks. ZNS rejects this model.

If ZNS stored state off-chain, the Mint would become a permanent hostage situation. If the Mint operator disappeared or suffered catastrophic hardware failure, the off-chain database would be lost. The on-chain Merkle root would be useless for reconstructing the namespace, rendering the protocol permanently dead.

ZNS forces every state transition into the strict 512-byte payload limit of an Orchard memo. The failure domain is Zcash consensus itself. If the Mint is vaporized, any developer can spin up a Resolver, scan the Zcash blockchain, and perfectly reconstruct the final registry state from the canonical history of Name Notes.

7.3 The Authorization Domain: OTP vs. Signatures

Zcash shielded addresses lack a standard message-signing protocol. Even if one existed, a raw cryptographic signature proves ownership of a key, but it does not prove temporal liveness, nor does it prevent replay attacks without a persistent on-chain nonce registry.

ZNS uses an in-band One-Time Passcode (OTP). The Mint encrypts 16 bytes of entropy and sends it to the Unified Address on record via a standard Zcash shielded transaction. Returning that exact payload to the Mint proves two things simultaneously: cryptographic read-access to the controller address, and temporal liveness bounded by the OTP expiration block.

The failure domain is key loss. If a user loses the viewing key to their Unified Address, they can never read the OTP. The name is permanently locked and cannot be updated or transferred until its registration interval lapses and the Mint returns it to the public pool.

8 Privacy Considerations

The public Name Note viewing key exposes registry contents to any scanner. The protocol does not provide name-binding confidentiality. Its payment-privacy claim is limited to the absence of a transparent transaction graph for ordinary shielded payments to the bound Unified Address. Network observers, the Mint, wallet providers, and authorization infrastructure can observe metadata outside that claim.

Out-of-scope privacy properties are recorded in Section **??**: registration privacy, resolution privacy, and freshness to a non-replaying client are not properties of this construction.

9 Deployment Parameters

The byte encodings and reducer do not determine which deployment an implementation follows. The initial active protocol manifest MUST fix the remaining deployment and policy values below.

Area	Required manifest value or referenced rule
Activation and genesis	Network identifier, activation height, Name Note viewing key, scan birthday, initial Mint account parameters, and initial recognized Mint note set.
Claim eligibility	Availability rule, price, payment asset, amount, payment recipient, required confirmations, and binding between payment and request.
Renewal	Renewal interval `renewal_interval` in blocks, refresh action, lapse rule, and renewal-fee policy.
Authorization transcript	Shielded receiver type, OTP relay memo grammar, OTP entropy source, OTP validity window ‘N‘ in blocks, OTP attempt limit, and one-time OTP consumption.
Attestation	Approved measurements, audited source revisions, immutable audit reports, verifier roots, minimum security versions, debug rejection, Mint-key binding, revocation, and key rotation.
TEE state	Sealed-state format, rollback detection, backup, restart, and compromise-recovery procedure.
Resolution	Response encoding, evidence encoding, confirmation policy, finality policy, and freshness limit.
ZNSIP release authority	Initial editor list, proposal repository, manifest publication location, and the decision process used to select a manifest for distribution.
Conformance corpus	Positive, negative, stale, conflicting, malformed, and reorganization vectors for every normative encoding and transition.

Each value is identified by the manifest content hash. A software release date or operator announcement is not a protocol activation rule.

10 ZNS Improvement Proposals

A source-code change cannot tell independent implementations when a new rule begins or which earlier bytes retain their old meaning. ZNS therefore

uses Zcash Name Service Improvement Proposals, abbreviated ZNSIPs, to specify protocol and process changes. The workflow follows the separation between publication, review, and deployment used by the ZIP process [6].

10.1 Proposal scope

One ZNSIP defines one change. A patch that affects only one implementation and does not affect interoperability does not require a ZNSIP. A change to any item below requires a Standards ZNSIP:

- canonical-name or wallet-resolution rules;
- memo, hash-transcript, field, or commitment encoding;
- candidate validity, Mint origin, lifecycle, ordering, or reducer behavior;
- authorization or attestation policy interpreted by another implementation;
- activation, manifest, or migration behavior; or
- a wallet or Resolver interface required for interoperability.

A Process ZNSIP changes the ZNSIP workflow or release process. An Informational ZNSIP records analysis without imposing a conformance requirement.

10.2 Required document fields

Every ZNSIP contains a number, title, owner list, status, category, creation date, license, discussion link, and dependency list. The editors assign numbers; an owner MUST NOT self-assign one. The repository filename is `znsip-NNNN.md`, where `NNNN` is the four-digit assigned number. The body contains Motivation, Specification, Rationale, Security Considerations, Privacy Considerations, Compatibility, Test Vectors, Reference Implementation, and Activation. A section that does not apply states why it does not apply.

A Standards ZNSIP that changes bytes or cryptographic computation MUST include positive and negative cross-language vectors. A Standards ZNSIP that changes trusted code, attestation, key custody, or authorization MUST identify the public security review required before activation. A Standards ZNSIP that changes state derivation MUST define migration, rollback, and behavior across the activation boundary.

10.3 Status workflow

The permitted statuses are `Draft`, `Review`, `Accepted`, `Active`, `Rejected`, `Withdrawn`, and `Superseded`.

`Draft`. The proposal may change without preserving interoperability.
`Review`. The owners assert that the specification and required vectors are complete.
`Accepted`. The editors have recorded the review decision and its unresolved objections. Acceptance does not activate the proposal.
`Active`. An active protocol manifest includes the proposal and its activation condition has been reached.
`Rejected`. The recorded decision declines the proposal.
`Withdrawn`. The owners have stopped pursuing the proposal.
`Superseded`. A later ZNSIP identifies the replacement proposal.

At least two editors maintain numbering, format, status records, and the versioned proposal repository. Editors MUST NOT use publication review to rewrite a proposal's technical decision. Every status change after `Draft` MUST identify the repository revision and a public decision record. The owners may move a proposal from `Draft` to `Review` after publishing the claimed-complete revision. The review period is at least 30 calendar days. After that period, at least two-thirds of the listed editors, and no fewer than two editors, must approve an `Accepted` or `Rejected` decision. An editor who owns the proposal does not vote on that decision and is excluded from the denominator.

10.4 Protocol manifests and activation

The encodings and reducer in this specification are portable across deployments. An implementation obtains its rule set from an *active protocol manifest* selected by the operator or software distribution. Manifest selection is a software-distribution trust decision; the Zcash chain does not select a ZNS manifest.

The manifest identifies all of these values:

1. a manifest identifier and predecessor manifest identifier;
2. the Zcash network and activation height;
3. every included Standards ZNSIP and its immutable content hash;
4. the Name Note viewing key, initial Mint-controlled note set, and scan

birthday;
5. accepted Mint identifiers and attestation policy artifacts;
6. the domain tags and protocol-version values used by the included rules;
7. the claim-eligibility, authorization, sealed-state, and resolution parameter values fixed by this deployment; and
8. hashes of the conformance vectors.

A Resolver MUST expose its manifest identifier and activation height with every resolution response. Two results computed under different manifests are not claims about the same protocol state. The manifest identifier is the lowercase hexadecimal digest of the exact published manifest bytes under unkeyed BLAKE2b with a 32-byte output and no personalization. Each ZNSIP content hash uses the same construction over the exact repository file bytes. A software release date or operator announcement is not a protocol activation rule.

An accepted Standards ZNSIP has no normative effect unless an active manifest includes its exact content hash. At activation height, the Resolver applies the new rules to occurrences at that height and later heights. It MUST apply the predecessor manifest to every earlier occurrence unless the ZNSIP defines an explicit migration from the already-derived state. A ZNSIP MUST NOT retroactively reinterpret an earlier occurrence by silence.

A memo or commitment change MUST use a new domain tag or an otherwise unambiguous version discriminator. A parser MUST NOT guess a version from malformed input. If a reorganization removes blocks at or above the activation height, the Resolver rolls back state under the rules that applied to each disconnected height and then replays the replacement chain under the same height boundary.

The initial publication of this process is ZNSIP-0000. The initial normative protocol specification is ZNSIP-0001.

11 Glossary

Terms defined in the body are gathered here for reference. Each is normative where it is defined; this section restates nothing.

Term	Definition or defining section
ZNS (Zcash Name Service)	The protocol this document specifies; see Section 1.
Zcash name	The wallet-facing string `n.zcash` for a canonical name `n`; see Section **??**.
Canonical name	The byte string from which a ZNS name is built; see Section **??**.
Name Note	An Orchard note whose memo carries one canonical ZNS tuple and whose note commitment is derived from the same tuple; see Section 3.
Action	One of `claim`, `update`, or `release`; see Section 3.
Predecessor (`prev_rcm`)	The 32-byte rcm_σ of the accepted prior Name Note for the same name, encoded as 64 lowercase hex characters; see Section 3.
Accepted tip	The most recently accepted Name Note for a given name at a canonical chain position; see Section **??**.
Binding (current binding)	The Unified Address in the accepted live tip, or none while released; see Section **??**.
Wallet	Requests a name action and completes the authorization exchange; see Section 2.
Mint	Performs the policy checks in Section **??** before creating a Name Note; runs inside a TEE; see Section 2.
Resolver	Performs the scan in Section **??** and applies the state reducer; see Section 2.
Attested Mint policy	Remote attestation binding enclave measurement, Mint authority, protocol version, network, and protected-state scheme; see Section 2.
Active protocol manifest	The deployment-selected bundle of protocol version, Zcash network, activation height, viewing key, Mint note set, and field values; see Section 10.
ZNSIP	A ZNS Improvement Proposal, processed per Section 10.
Name Note viewing key	The published Orchard full viewing key of the Name Note account; see Section 1.
One-time passcode (OTP)	A 16-byte random value issued by the Mint to a controller receiver and returned by the wallet to authorize an `update` or `release`; see Section 5.
OTP relay memo	The Orchard shielded memo `ZNS:otp:<name>:<verb>:<ua>:<otp>` carrying an OTP to the controller receiver; see Section 5.

External terms inherited from the Zcash protocol specification [1] or its ZIPs are not redefined here: *Orchard action*, *Orchard memo*, *note commitment* (`cmx`), *Unified Address*, and *incoming viewing key* retain the meaning fixed by their normative sources [3, 4, 1].

References

[1] Electric Coin Company and Zcash Foundation. *Zcash Protocol Specification*. https://zips.z.cash/protocol/protocol.pdf.

[2] Zcash Improvement Proposal 212. *Allow Recipient to Derive Ephemeral Secret from Note Plaintext*. https://zips.z.cash/zip-0212.

[3] Zcash Improvement Proposal 224. *Orchard Shielded Protocol*. https://zips.z.cash/zip-0224.

[4] Zcash Improvement Proposal 316. *Unified Addresses and Unified Viewing Keys*. https://zips.z.cash/zip-0316.

[5] S. Bradner and J. Leiba. *Key words for use in RFCs to Indicate Requirement Levels*. BCP 14, RFC 2119 and RFC 8174. https://www.rfc-editor.org/info/bcp14.

[6] Zcash Improvement Proposal 0. *ZIP Process*. https://zips.z.cash/zip-0000.

[7] zcashme. *zns-mint reference implementation*. https://github.com/zcashme/zns-mint.